TRANSACTION FRAUD DETECTION

FINAL PYTHON PROJECT

JULIE MIQUÉLAJAURÉGUI, ECATERINA GUPCA, NOANN MANON-MAZA

TABLE OF CONTENTS

1. INTRODUCTION

1.1 PROBLEM STATEMENT

Credit card fraud causes significant financial losses every year for both financial institutions and individual cardholders. Fraudulent transactions represent only a tiny fraction of all transactions processed, which makes them extremely difficult to detect using manual review or simple rule-based systems. This project addresses the problem of automatically identifying fraudulent credit card transactions from transaction data using supervised machine learning.

1.2 MOTIVATION

As digital payments continue to grow, so does the volume and sophistication of fraud attempts. Manual fraud detection does not scale, and naive approaches (such as flagging all high-value transactions) generate too many false alarms while still missing genuine fraud. Machine learning offers a way to learn subtle statistical patterns that distinguish fraudulent from legitimate behaviour, updating automatically as more data becomes available. Beyond the direct financial motivation, this project is also an opportunity to practice a complete, realistic data science workflow on a strongly imbalanced, real-world dataset — from cleaning and exploration through to model evaluation with business-relevant metrics.

1.3 TARGET QUESTION

Can a machine learning model, trained on anonymized transaction features, reliably distinguish fraudulent credit card transactions from legitimate ones despite the fraud class representing less than 0.2% of all transactions?

1.4 EXPECTED OUTCOME

We expect that tree-based ensemble models (Random Forest, XGBoost) will outperform a simple linear baseline (Logistic Regression) in terms of precision and F1-score, because fraud patterns are likely non-linear combinations of the anonymized features. We also expect that, without addressing class imbalance, all models would be biased toward predicting the majority (legitimate) class, so a resampling strategy such as SMOTE is necessary to obtain a model that meaningfully detects fraud. The final expected outcome is a comparative evaluation of three models, leading to a recommendation of the model that offers the best trade-off between catching fraud (recall) and minimizing false alarms (precision).

2. DATASET DESCRIPTION

This project uses the "Credit Card Fraud Detection" dataset, publicly available on Kaggle:

https://www.kaggle.com/datasets/mlg-ulb/creditcardfraud

The dataset was originally collected and made available by the Machine Learning Group of Université Libre de Bruxelles (ULB), in collaboration with Worldline, as part of a research collaboration on big data mining and fraud detection.

2.1 SIZE AND FORMAT

- File format: single CSV file (creditcard.csv)
- Size: 284,807 transactions (rows) × 31 columns
- Transactions made by European cardholders over two days in September 2013

2.2 FEATURES

- Time: number of seconds elapsed between each transaction and the first transaction in the dataset
- V1 to V28: 28 numerical features obtained via a Principal Component Analysis (PCA) transformation of the original features, which are not disclosed for confidentiality reasons
- Amount: transaction amount, which can be used for cost-sensitive analysis
- Class: target variable — 1 for fraud, 0 for legitimate transaction

2.3 TARGET VARIABLE DISTRIBUTION

The dataset is extremely imbalanced: only 492 transactions (0.173%) are fraudulent out of 284,807 total transactions, while 284,315 (99.827%) are legitimate.

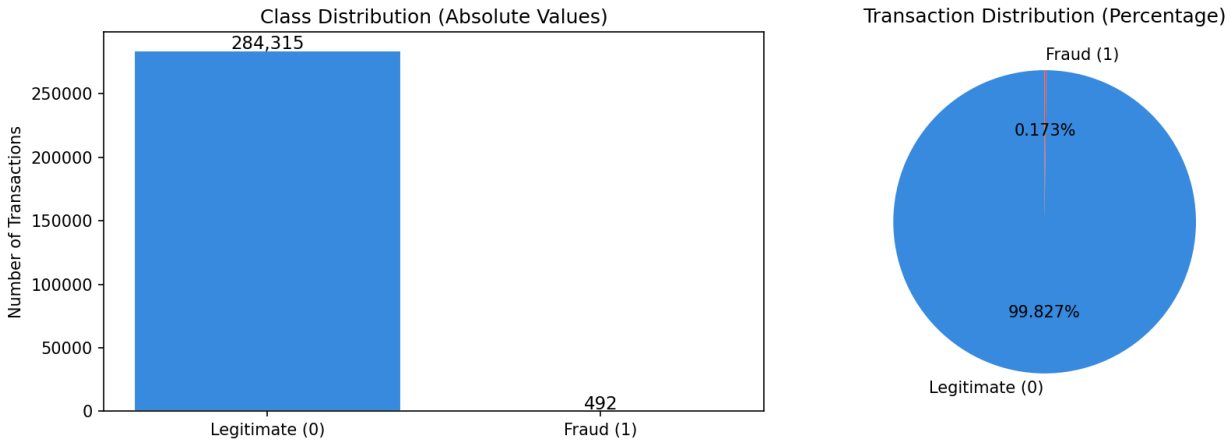

Figure 1 — Class distribution: raw counts and percentage split between legitimate and fraudulent transactions.

2.4 LICENSE AND USAGE CONDITIONS

The dataset is released under the Open Database License (ODbL) on Kaggle, and it is intended for research and educational purposes. Commercial use of the original raw transaction data is restricted, and no personally identifiable cardholder information is included since all sensitive features were anonymized via PCA before release.

2.5 DATASET LIMITATIONS

- Because features V1–V28 are anonymized PCA components, individual feature meaning cannot be interpreted directly (e.g. we cannot say V14 represents “location” or “merchant type”), which limits explainability.
- Data covers only two days of transactions from European cardholders, so patterns may not generalize to other regions, time periods, or currencies.
- Fraud patterns evolve over time (concept drift); a model trained on 2013 data may not capture more recent fraud techniques.

- The severe class imbalance means very few fraud examples are available to learn from, increasing the risk of overfitting to the specific fraud cases present in the dataset.

3. PREPROCESSING

3.1 MISSING VALUES

The dataset was checked column by column for missing values; none were found (0 missing values across all 31 columns), so no imputation was necessary.

3.2 DUPLICATES

1,081 exact duplicate rows were identified and removed, reducing the dataset from 284,807 to 283,726 rows. Duplicate transactions could otherwise bias the model by over-representing certain patterns and could also leak identical rows across the train/test split.

3.3 DATA TYPE VERIFICATION

All feature columns (Time, V1–V28, Amount) were confirmed to be numerical (float64), and the target Class was confirmed to be strictly binary (values 0 and 1 only, integer type). No categorical encoding was required since all inputs were already numeric.

3.4 OUTLIER ANALYSIS

An IQR-based outlier analysis was performed on the Amount feature (Q1 = €5.60, Q3 = €77.51, IQR = €71.91). This identified 31,685 outlier transactions (11.17% of the cleaned dataset), of which 87 were actual frauds and 31,598 were legitimate. Since unusually large or small amounts are directly relevant to identifying fraud, these outliers were deliberately kept in the dataset rather than removed.

3.5 FEATURE SCALING

The Time and Amount columns were the only features not already standardized (V1–V28 are outputs of a PCA transformation and are already on comparable scales). Both columns were standardized using scikit-learn's StandardScaler, fitted only on the training set and then applied to the test set, in order to avoid data leakage from the test set into the scaling parameters.

3.6 TRAIN/TEST SPLIT

The cleaned dataset was split into 80% training and 20% testing sets using a stratified split (stratify=y, random_state=42) to preserve the original fraud ratio in both subsets:

Set	Rows	Fraud cases	Fraud %
Training set	226,980	378	0.167%
Test set	56,746	95	0.167%

3.7 HANDLING CLASS IMBALANCE (SMOTE)

Because only 0.167% of the training data is fraud, models trained directly on this data would strongly favor predicting "legitimate" for every transaction. To address this, SMOTE (Synthetic Minority Over-sampling Technique) was applied to the scaled training set only (never to the test set, to keep evaluation realistic). SMOTE generates synthetic fraud examples by interpolating between existing minority-class neighbours, bringing the training set to a 50/50 balance (226,602 legitimate vs. 226,602 synthetic + real fraud examples, 453,204 rows total).

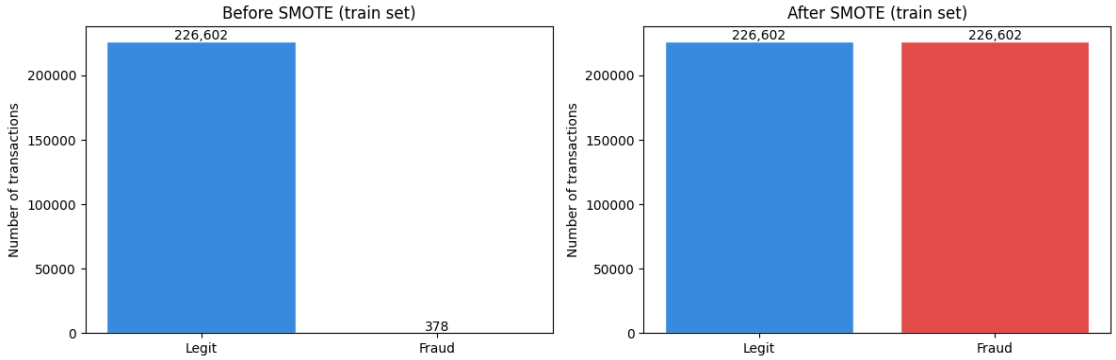

Figure 2 — Class balance in the training set before and after SMOTE resampling.

4. DATA VISUALIZATION AND EDA

4.1 TRANSACTION TIMING AND AMOUNT DISTRIBUTIONS

Transactions are distributed across the full 48-hour window of the dataset, with visible dips corresponding to nighttime hours. Transaction amounts are heavily right-skewed, with most transactions being small (a median around €22) and a long tail of larger purchases.

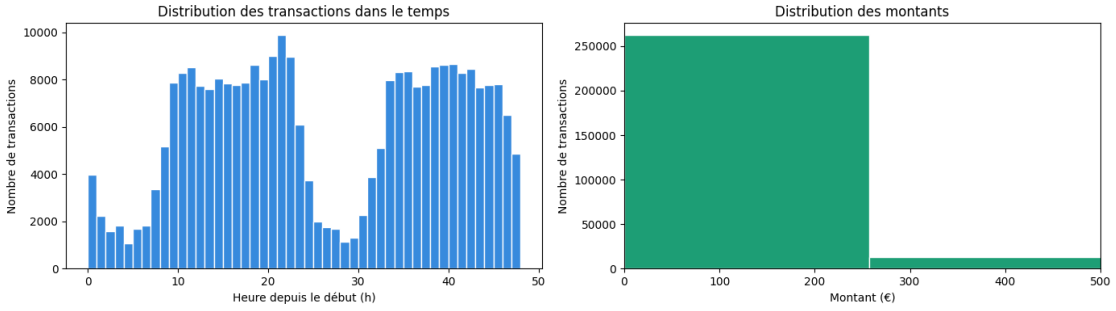

Figure 3 — Distribution of transaction time (hours since start) and transaction amount.

4.2 TRANSACTION AMOUNT: FRAUD VS. LEGITIMATE

Fraudulent transactions have a slightly higher mean amount (€122.21) than legitimate ones (€88.29), but a much lower median (€9.25 vs. €22.00), suggesting fraud tends to involve many small "test" transactions alongside occasional larger ones.

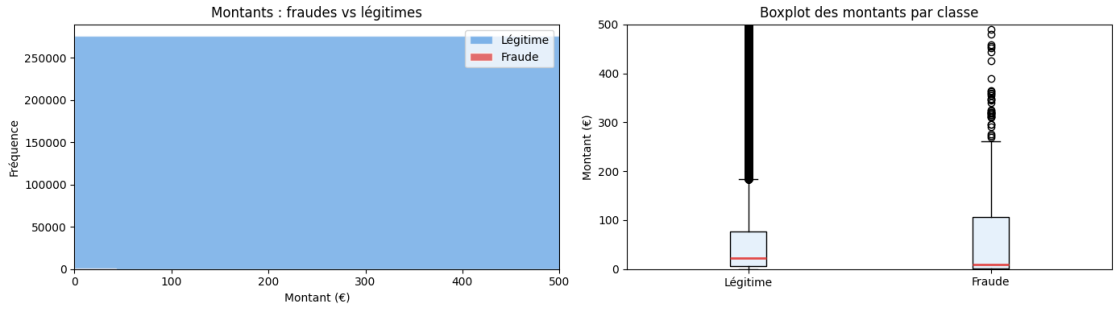

Figure 4 — Transaction amount distribution and boxplot, compared between legitimate and fraudulent transactions.

4.3 CORRELATION WITH THE TARGET VARIABLE

Correlations between each feature and Class were computed. The most negatively correlated features are V17 (–0.31), V14 (–0.29), V12 (–0.25), V10 (–0.21) and V16 (–0.19); the most positively correlated is V11 (0.15). No feature is correlated in isolation with fraud strongly enough to be used alone — this supports the need for a multivariate model rather than simple thresholding rules.

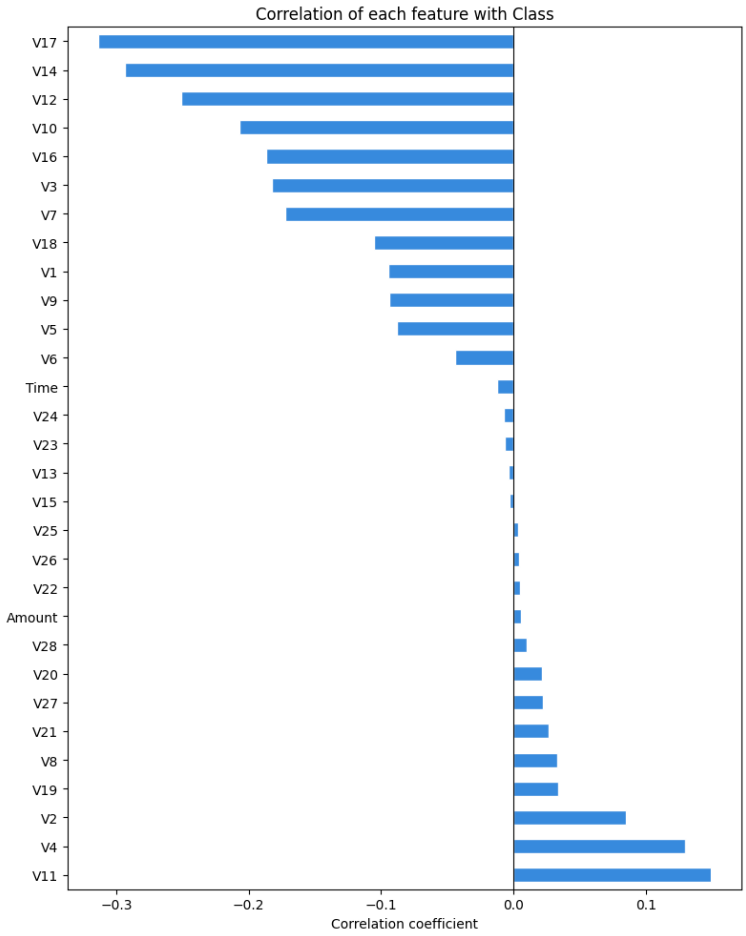

Figure 5 — Correlation of each feature with the Class target variable.

4.4 MOST DISCRIMINATIVE FEATURES

Boxplots of the five features most correlated with fraud (V17, V14, V12, V10, V16) show a clear separation in medians and spread between the legitimate and fraud classes, confirming these are the most informative individual predictors.

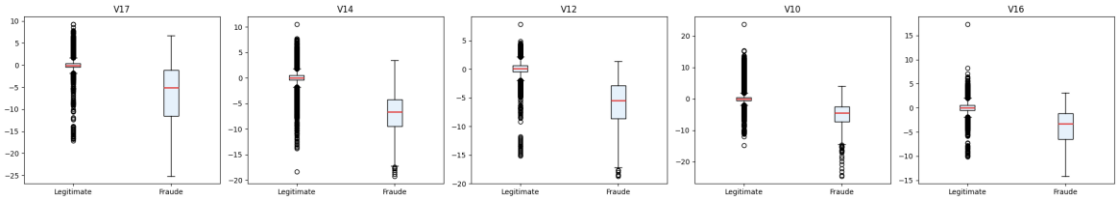

Figure 7 — Distribution of the top 5 most fraud-correlated features, compared by class.

4.5 FRAUD BY HOUR OF DAY

When transactions are grouped by hour of day, fraud is proportionally more frequent during late-night and early-morning hours, when overall transaction volume is lower and cardholders are less likely to notice unusual activity immediately.

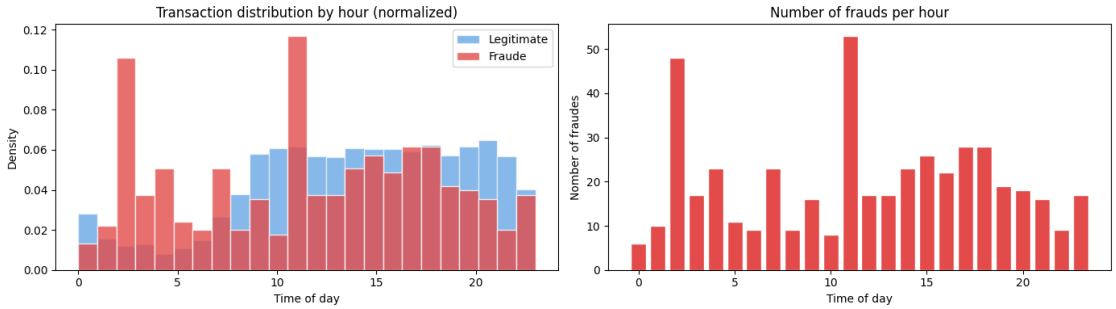

Figure 8 — Distribution of transactions by hour of day (normalized) and count of frauds per hour.

5. FEATURE ENGINEERING

Because the 28 principal predictive features (V1–V28) are already the output of a PCA transformation performed by the dataset providers, extensive manual feature engineering was neither necessary nor really possible without access to the original raw features. The feature engineering work in this project focused on preparation rather than creation of new predictive signals:

- Hour extraction: an Hour feature (0–23) was derived from Time for exploratory analysis of fraud patterns across the day.
- Scaling: Time and Amount were standardized (zero mean, unit variance) with StandardScaler so that they are on a comparable scale to the already-standardized V1–V28 features; this matters in particular for Logistic Regression, which is sensitive to feature scale.
- No dimensionality reduction was applied beyond the PCA already present in the raw data, since 28 components is manageable for all three chosen algorithms and further reduction risked discarding the very components most correlated with fraud (e.g. V14, V17).
- No categorical encoding was needed since every feature is already numeric.

- Class balancing via SMOTE (described in Section 3.7) can also be considered a form of data-level feature engineering aimed at reshaping the training distribution rather than the feature space itself.

6. MACHINE LEARNING MODEL TRAINING

6.1 ALGORITHMS USED

- Logistic Regression — a simple linear classifier used as an interpretable baseline.
- Random Forest — an ensemble of decision trees that can capture non-linear relationships and feature interactions.
- XGBoost — a gradient boosting ensemble that builds trees sequentially, generally very effective on imbalanced tabular data.

6.2 MODEL WORKFLOW

For each model, the workflow was: (1) train on the SMOTE-balanced, scaled training set (X_train_balanced, y_train_balanced); (2) generate predictions and predicted probabilities on the untouched, unbalanced, scaled test set (X_test_scaled), which reflects the real-world 0.167% fraud rate; (3) evaluate using classification metrics appropriate for imbalanced data (Section 7). Training only on balanced data while testing on realistic, unbalanced data ensures the reported performance reflects how the model would behave in production.

6.3 TRAIN/TEST SPLIT

As described in Section 3.6, an 80/20 stratified split was used, with SMOTE applied only after the split and only to the training portion, to avoid any leakage of synthetic or duplicated information into the test set.

6.4 HYPERPARAMETERS

Baseline hyperparameters were used for all three models, with random_state=42 fixed throughout for reproducibility:

Model	Key parameters
Logistic Regression	max_iter=1000, random_state=42 (default L2 regularization)
Random Forest	n_estimators=100, n_jobs=-1, random_state=42
XGBoost	eval_metric='logloss', n_jobs=-1, random_state=42

These were left at their default/baseline settings for this iteration of the project; hyperparameter tuning (e.g. grid search on tree depth, regularization strength, or learning rate) is identified as a natural next step in Section 10.

6.5 REASON FOR CHOOSING THESE MODELS

Logistic Regression was included as a fast, interpretable baseline that establishes a reference point. Random Forest and XGBoost were chosen because they can capture non-linear decision boundaries and complex feature interactions, which is important here since no single feature separates fraud from legitimate transactions cleanly (Section 4.3). XGBoost in particular is widely used in industry for fraud detection because of its strong performance on structured/tabular data and its efficiency at scale.

Note: Class itself is a categorical (binary) label, so this problem is naturally framed as classification rather than regression — the goal is to assign each transaction to one of two discrete categories (fraud / legitimate) and to produce a class probability, not to predict a continuous numeric value. This also explains why classification-specific tools such as SMOTE and precision/recall metrics apply here, whereas they would have no equivalent in a regression setting.

7. EVALUATION METRICS AND RESULTS

7.1 WHY NOT ACCURACY ALONE?

With only 0.173% fraud, a trivial model that always predicts "legitimate" would already achieve about 99.8% accuracy while being completely useless for fraud detection. For this reason, the following metrics were used instead:

- Precision — of the transactions flagged as fraud, how many are actually fraud (avoids blocking innocent customers)
- Recall — of the actual frauds, how many were detected (avoids letting fraud through undetected)
- F1-score — harmonic mean balancing precision and recall
- AUC-ROC — the model's ability to rank fraud above legitimate transactions across all possible decision thresholds

7.2 RESULTS SUMMARY

Model	Precision	Recall	F1-score	AUC-ROC
Logistic Regression	0.0530	0.8737	0.1000	0.9619
Random Forest	0.9114	0.7579	0.8276	0.9656
XGBoost	0.7212	0.7895	0.7538	0.9687

7.3 CONFUSION MATRICES

The confusion matrices below show, for each model, the count of true negatives, false positives, false negatives, and true positives on the 56,746-transaction test set (95 actual frauds).

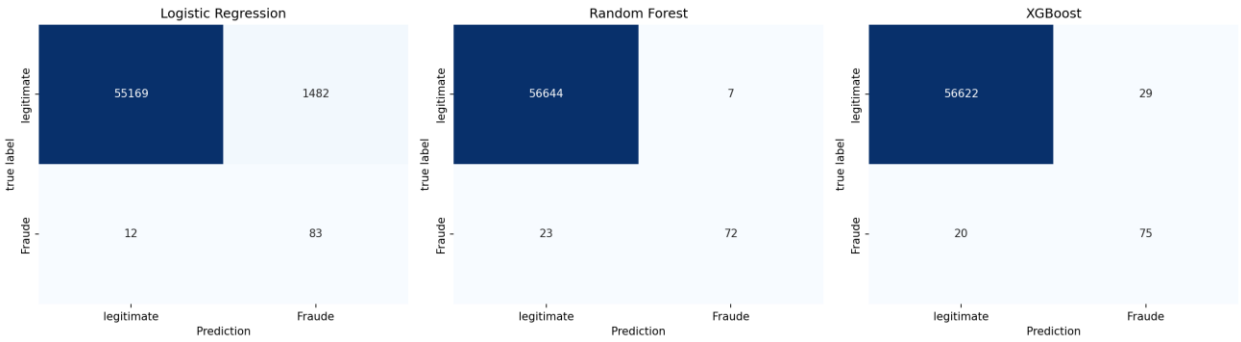

Figure 9 — Confusion matrices for Logistic Regression, Random Forest, and XGBoost.

7.4 ROC CURVES

All three models achieve a high AUC-ROC (above 0.96), meaning that overall ranking ability is strong for all of them; the differences show up mainly in precision and recall at the chosen classification threshold rather than in raw discriminative power.

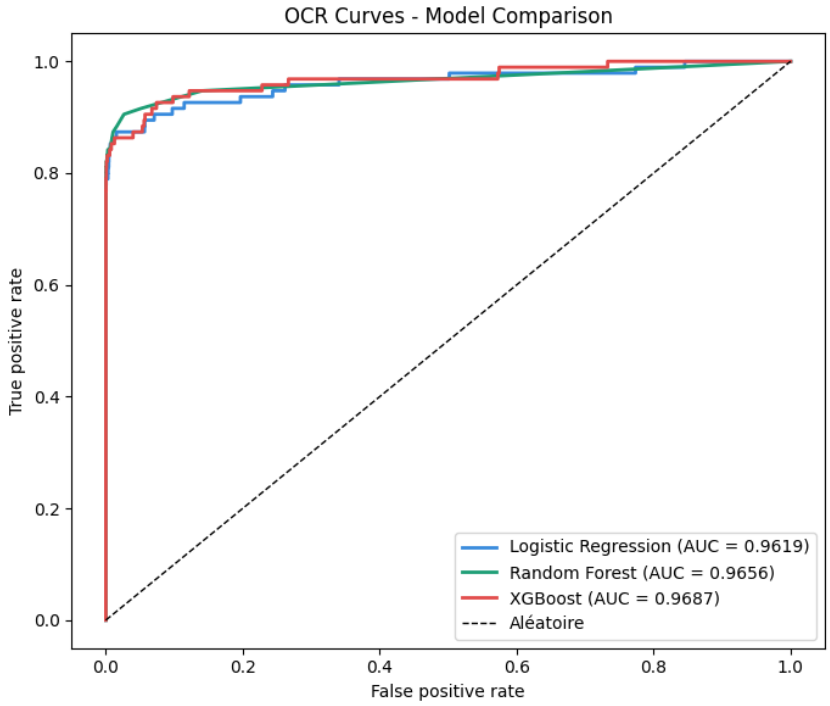

Figure 10 — ROC curves comparing all three models, with AUC values.

7.5 INTERPRETATION, STRENGTHS AND WEAKNESSES

Logistic Regression achieves the highest recall (87.4%), catching the most actual frauds, but its precision is extremely low (5.3%) meaning that for every real fraud it correctly flags, it also raises a very large number of false alarms on legitimate transactions. This makes it impractical to deploy alone in a real system, since it would block or flag far too many genuine customers.

Random Forest achieves the best precision (91.1%) and the best F1-score (0.83), meaning that when it flags a transaction as fraud, it is right the vast majority of the time, at some cost to recall (75.8% of frauds caught).

XGBoost sits between the two on precision/recall (72.1% precision, 79.0% recall) and achieves the highest AUC-ROC (0.969), indicating the best overall separation between the two classes; with threshold tuning it could likely be adjusted to match or beat Random Forest's F1-score while keeping a good recall.

Overall, Random Forest is the strongest candidate if minimizing false alarms is the priority (e.g. to avoid annoying legitimate customers), while XGBoost offers the best balance and the most headroom for improvement through threshold and hyperparameter tuning. Logistic Regression, despite its weak standalone performance, remains useful as an interpretable baseline and a sanity check for the more complex models.

8. RESPONSIBLE AI PRACTICES

8.1 BIAS AND FAIRNESS

Because features V1–V28 are anonymized PCA components, it is not possible to check directly for bias against protected attributes such as gender, age, or nationality, since these attributes are not present or identifiable in the dataset. This is partly a privacy safeguard, but it also means fairness cannot be audited along those dimensions with this dataset alone; any real deployment would need access to (and fairness testing against) such attributes through a separate, carefully governed process.

8.2 PRIVACY

The dataset providers anonymized the original transaction features via PCA specifically to protect cardholder identity and sensitive transaction details, and no directly identifying information (names, card numbers, merchant identities) is included. This project only uses this already-anonymized public dataset and does not attempt any re-identification.

8.3 SOURCE RELIABILITY

The dataset is a well-established, widely cited benchmark (ULB Machine Learning Group / Worldline, hosted on Kaggle) commonly used in academic fraud-detection research, which supports its reliability for educational purposes. However, it reflects only two days of 2013 European transactions, which limits how representative it is of current, global fraud patterns (see Section 2.5).

8.4 ETHICAL RISKS

- False positives have a real human cost: legitimate customers whose transactions are wrongly blocked may face inconvenience, embarrassment, or financial disruption — this is a key reason precision was tracked alongside recall rather than optimizing for recall alone.
- False negatives (missed fraud) have a direct financial cost to cardholders and institutions, and repeated undetected fraud could erode trust in the payment system.
- A model trained on a specific historical window (2013) risks becoming stale as fraud techniques evolve (concept drift), so any production use would require ongoing monitoring and retraining.

8.5 AVOIDING MISLEADING CLAIMS

The team deliberately avoided reporting accuracy as the headline metric, since with 99.8% legitimate transactions it would misleadingly suggest near-perfect performance even for a useless model. Precision, recall, F1-score, and AUC-ROC were reported together, and both strengths and weaknesses of each model are stated explicitly (Section 7.5) rather than presenting only the best-looking metric for each model.

9. CONCLUSION AND FUTURE WORK

9.1 MAIN FINDINGS

- All three models achieved strong overall discriminative ability (AUC-ROC ≥ 0.96), confirming that fraudulent transactions do carry a learnable statistical signature within the PCA-transformed features.
- SMOTE was necessary to obtain usable fraud recall from any model; without addressing the 0.167% imbalance, models would default toward always predicting "legitimate."
- Tree-based ensembles (Random Forest, XGBoost) clearly outperformed the linear baseline on precision and F1-score, confirming our initial hypothesis that fraud patterns involve non-linear feature interactions.
- There is a clear precision/recall trade-off across models: Logistic Regression favors recall at the cost of precision, Random Forest favors precision, and XGBoost offers the most balanced trade-off with the best AUC-ROC.

9.2 WHAT THE GROUP LEARNED

Working through this project reinforced why accuracy is a misleading metric for imbalanced classification, and gave hands-on experience with a full, realistic pipeline: cleaning, EDA, correct train/test/SMOTE ordering (to avoid data leakage), model comparison, and metric selection tailored to a business problem rather than a generic one.

9.3 POSSIBLE IMPROVEMENTS AND NEXT STEPS

- Hyperparameter tuning (grid or random search) for Random Forest and XGBoost, particularly tree depth, learning rate, and class-weighting, to push F1-score higher.
- Threshold tuning: adjusting the probability cutoff (rather than the default 0.5) for XGBoost and Random Forest to better balance precision and recall for a specific business cost trade-off.
- Trying alternative imbalance techniques (e.g. class weighting, ADASYN, or undersampling) as a comparison to SMOTE.
- Adding cost-sensitive evaluation, e.g. estimating the monetary impact of false positives (customer friction) vs. false negatives (fraud losses), using the Amount feature.
- Testing model robustness on more recent or more diverse transaction data to check for concept drift, since this dataset only covers two days in 2013.

10. TEAM CONTRIBUTION

The table below summarizes each member's contribution across the main areas of the project.

Team member	Coding	Analysis	Report writing	Slides	Presentation
Noann	Data cleaning, preprocessing	EDA, correlation analysis	Sections 2–4	Slides 1–3	Intro & EDA
Julie	Model training	Metric interpretation	Sections 6–8	Slides 4–6	Models & results
Ecaterina	Evaluation, plots	Responsible AI review	Sections 1, 9–10	Slides 7–9	Conclusion & Q&A